Text Classification with spaCy I

In this chapter, we're going to cover the following main topics :

- Understanding the basics of text classification
- Training the spaCy text classifier
- Sentiment analysis with spaCy

This chapter requires the following Python libraries :

- TensorFlow >=2.2.0
- NumPy

Text Classification with spaCy II

- pandas
- matplotlib

Text classification is the task of assigning a set of predefined labels to text. Given a set of predefined classes and some text, you want to understand which predefined class this text falls into. We have to determine the classes ourselves by the nature of our data before starting the classification task. For example, a customer review can be positive, negative, or neutral.

Text Classification with spaCy III

Text classifiers are used for detecting spam emails in your mailbox, determining the sentiment of customer's reviews, understanding customer's intent, sorting customer's complaint tickets, and so on.

Text classification is a supervised machine learning task. It means that the classifier can predict the class label of a text based on example input text-class label pairs. Hence, to train a text classifier, we need a labeled dataset. A labeled dataset is basically a list of text-label pairs.

Text Classification with spaCy IV

Text classification can also be divided into three categories depending on the number of classes used :

- Binary text classification means that we want to categorize our text into two classes.
- Multiclass text classification means that there are more than two classes. Each class is mutually exclusive ? one text can belong to one class only. Equivalently, a training instance can be labeled with only one class label. An example is rating

Text Classification with spaCy V

customer reviews. A review can have 1, 2, 3, 4, or 5 stars (each star category is a class).

- Multilabel text classification is a generalization of multiclass classification, where multiple labels can be assigned to each example text. For example, classifying toxic social media messages is done with multiple labels. This way, our model can distinguish different levels of toxicity. Class labels are typically toxic, severe toxic, insult, threat, obscenity. A message can include both insult, toxicity, and obscenity, and so on.

Training the spaCy text classifier I

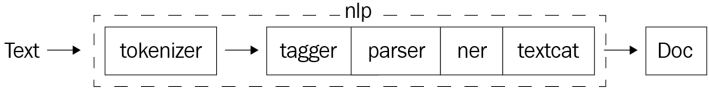
In this section, we will learn about the details of spaCy's text classifier component TextCategorizer. In the preceeding Core Operations with spaCy Chapter, we saw that the spaCy NLP pipeline consists of components. In the Linguistic Features Chapter, we learned about the essential components of the spaCy NLP pipeline, which are the sentence tokenizer, POS tagger, dependency parser, and named entity recogition (NER).

TextCategorizer is an optional and trainable pipeline component. In order to train it, we need to provide examples and their class labels. We first add TextCategorizer to the NLP pipeline and then do the training procedure. The Figure below shows where exactly

Training the spaCy text classifier II

the TextCategorizer component lies in the NLP pipeline; this component comes after the essential components. In the following diagram, textcat refers to the TextCategorizer component.

A neural network architecture lies behind spaCy's TextCategorizer.



The TextCategorizer class I

TextCategorizer provides us with user-friendly and end-to-end approaches to train the classifier, so we don't have to deal directly with the neural network architecture.

First of all, we import TextCategorizer from the pipeline components :

```
from spacy.pipeline.textcat import DEFAULT_SINGLE_TEXTCAT_MODEL
```

TextCategorizer is available in two types, single-label classifier and multilabel classifier. A multilabel classifier can predict more than one class. A single-label classifier predicts only one class for each example and classes are mutually exclusive. The preceding import line imports the single-label classifier and the following code imports the multilabel classifier :

```
from spacy.pipeline.textcat_multilabel import DEFAULT_MULTI_
TEXTCAT_MODEL
```

Next, we need to provide a configuration to the TextCategorizer component. We provide two parameters here, a threshold value and a model name (either Single or Multi depending on the classification task). TextCategorizer internally generates a probability

for each class and a class is assigned to the text if the probability of this class is higher than the threshold value.

A traditional threshold value for text classification is 0.5, however, if you want to make a prediction with higher confidence, you can make the threshold higher, such as 0.6, 0.7, or 0.8.

Putting it altogether, we can add a single-label TextCategorizer component to the nlp pipeline as follows :

```
from spacy.pipeline.textcat import DEFAULT_SINGLE_TEXTCAT_MODEL
config = {
  "threshold": 0.5,
  "model": DEFAULT_SINGLE_TEXTCAT_MODEL
}
```

```
6  textcat = nlp.add_pipe("textcat", config=config)
```

Adding a multilabel component to the nlp pipeline is similar :

```
1  from spacy.pipeline.textcat_multilabel import
2  DEFAULT_MULTI_TEXTCAT_MODEL
3  config = {
4    "threshold": 0.5,
5    "model": DEFAULT_MULTI_TEXTCAT_MODEL
6  }
7  textcat = nlp.add_pipe("textcat_multilabel", config=config)
```

Formatting training data for the TextCategorizer I

Let's start our code by preparing a small training set. We'll prepare a customer sentiment dataset for binary text classification. The label will be called sentiment and can obtain two possible values, 0 and 1 corresponding to negative and positive sentiment. The following training set contains 6 examples, 3 being positive and 3 being negative :

```
train_data = [
("I loved this product, very easy to use.", {"cats":
  {"sentiment": 1}}),
("I'll definitely purchase again. I recommend this
product.", {"cats": {"sentiment": 1}}),
("This is the best product ever. I loved the scent and the
feel. Will buy again.", {"cats": {"sentiment": 1}}),
("Disappointed. This product didn't work for me at all",
{"cats": {"sentiment": 0}}),
("I hated the scent. Won't buy again", {"cats":
  {"sentiment": 0}}),
("Truly horrible product. Very few amount of product for a
high price. Don't recommend.", {"cats": {"sentiment": 0}})
```

```
14  ]
```

Each training example is a tuple of a text and a nested dictionary. The dictionary contains the class label in a format that spaCy recognizes. The "cats" field means the categories. Then we include the class label sentiment and its value. The value should always be a floating point number.

In the code, we will introduce the class label we choose to the TextCategorizer component. Let's see the complete code. First, we do the necessary imports :

```
1  import random
2  import spacy
3  from spacy.training import Example
4  from spacy.pipeline.textcat import DEFAULT_SINGLE_TEXTCAT_MODEL
```

In the last line of the code block, we imported a text categorizer model. Next, we'll do the pipeline and TextCategorizer component initialization :

```
nlp = spacy.load("en_core_web_md")
config = {
  "threshold": 0.5,
  "model": DEFAULT_SINGLE_TEXTCAT_MODEL
}
textcat = nlp.add_pipe("textcat", config=config)
```

Now, we'll do some work on the newly created TextCategorizer component, textcat. We'll introduce our label sentiment to the TextCategorizer componenet by calling add_label. Then, we need to initialize this component with our examples. This step is different than what we did in the NER training code in the Customizing spaCy Models Chapter.

Indeed NER is an essential component, hence it's initialized by the pipeline always.

TextCategorizer is an optional component, and it comes as a blank statistical model.

The following code adds our label to the TextCategorizer component and then initializes the TextCategorizer model's weights with the training examples :

```
1  textcat.add_label("sentiment")
2  train_examples = [Example.from_dict(nlp.make_doc(text), label)
3  for text,label in train_data]
4  textcat.initialize(lambda: train_examples, nlp=nlp)
```

Note that we feed the examples to textcat.initialize as Example objects.

Defining the training loop I

We're ready to define the training loop. First of all, we'll disable other pipe components so that only textcat will be trained. Second, we will create an optimizer object by calling resume_training, keeping the weights of the existing statistical models.

For each epoch, we go over training examples one by one and update the weights of textcat. We go over the data for 20 epochs. The following code defines the training loop :

```
epochs=20
with nlp.select_pipes(enable="textcat"):
optimizer = nlp.resume_training()
for i in range(epochs):
random.shuffle(train_data)
for text, label in train_data:
```

```
doc = nlp.make_doc(text)
example = Example.from_dict(doc, label)
nlp.update([example], sgd=optimizer)
```

With this relatively short code segment, we trained a text classifier ! Here's the output

```
{'textcat': 0.25}
{'textcat': 0.25403276085853577}
{'textcat': 0.23483306169509888}
{'textcat': 0.233804851770401}
{'textcat': 0.2132250368595123 3}
{'textcat': 0.19576627016067505}
{'textcat': 0.17215843498706818}
{'textcat': 0.17311060428619385}
{'textcat': 0.128489151597023}
{'textcat': 0.13611891865730286}
{'textcat': 0.11497252434492111}
{'textcat': 0.07584841549396515}
{'textcat': 0.05945904180407524}
{'textcat': 0.07257045060396194}
{'textcat': 0.03416129946708679}
{'textcat': 0.04692266136407852}
{'textcat': 0.019232971593737602}
{'textcat': 0.029277725145220757}
{'textcat': 0.010624242015182972}
```

Sentiment analysis with spaCy I

In this section, we'll work on a real-world dataset and train spaCy's TextCategorizer on this dataset. We'll be working on the Amazon Fine Food Reviews dataset (https ://www.kaggle.com/fine-food-reviews) from Kaggle. The original dataset is huge, with 100,000 rows. We sampled 4,000 rows. This dataset contains customer reviews about fine food sold on Amazon. Reviews include user and product information, user rating, and text.

Exploring the dataset I

Let's do our dataset exploration step by step : First, we'll do the imports for reading and visualizing the dataset :

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

Then, we'll read the CSV file into a pandas DataFrame and output the shape of the DataFrame :

```
reviews_df=pd.read_csv('data/Reviews.csv')
reviews_df.shape
(3999, 10)
```

Next, we examine the rows and the columns of the dataset by printing the first 10 rows :

```
reviews_df.head()
```

The resulting view tells us there are 10 rows including the review text and the review

score.

We'll be using the Text and Score columns ; hence, we'll drop the other columns that we won't use. We ?ll also call the dropna() method to drop the rows with missing values :

```
reviews_df = reviews_df[['Text','Score']].dropna()
```

We can have a quick look at how the review scores are distributed :

```
ax=reviews_df.Score.value_counts().plot(kind='bar',
colormap='Paired')
plt.show()
```

To balance the classes, we'll combine 1,2,3 stars and 4,5 stars. We'll treat ratings of 1, 2 and 3 stars as negative, and ratings over 4 stars as positive. The following code segment assigns a negative label to all reviews with a rating below 4 stars and a positive label to

all reviews with a rating above 4 stars :

```
reviews_df.Score[reviews_df.Score<=3]=0
reviews_df.Score[reviews_df.Score>=4]=1
```

Plot the distribution of the ratings again :

```
ax=reviews_df.Score.value_counts().plot(kind='bar',
colormap='Paired')
plt.show()
```

Training the TextClassifier component I

Now, we're ready to start the training procedure. We'll train a binary text classifier with the multilabel classifier this time. Again, let's go step by step : We start by importing the spaCy classes as follows :

```
import spacy
import random
from spacy.training import Example
from spacy.pipeline.textcat_multilabel import DEFAULT_
MULTI_TEXTCAT_MODEL
```

Next, we'll create a pipeline object, nlp, define the classifier configuration, and add the TextCategorizer component to nlp with the following configuration :

```
nlp = spacy.load("en_core_web_md")
config = {
  "threshold": 0.5,
  "model": DEFAULT_MULTI_TEXTCAT_MODEL
}
```

```
6  textcat = nlp.add_pipe("textcat_multilabel",
7  config=config)
```

3. After creating the text classifier component, we'll convert training sentences and ratings into a spaCy usable format. We'll iterate every row of the DataFrame with iterrows(), and for each row we'll extract the Text and Score fields. Then, we'll create a spaCy Doc object from the review text and make a dictionary of the class labels as well. Finally, we will create an Example object and append it to the list of training examples :

```
1  train_examples = []
2  for index, row in reviews_df.iterrows():
3  text = row["Text"]
4  rating = row["Score"]
5  label = {"POS": True, "NEG": False} if rating == 1
```

```
  else {"NEG": True, "POS": False}
  train_examples.append(Example.from_dict(nlp.make_
  doc(text), {"cats": label}))
```

We'll use POS and NEG labels for positive and negative sentiment, respectively. We'll introduce these labels to the new component and also initialize the component with examples :

```
textcat.add_label("POS")
textcat.add_label("NEG")
textcat.initialize(lambda: train_examples, nlp=nlp)
```

We're ready to define the training loop ! We went over the training set for two epochs, but you can go over more if you like. The following code snippet will train the new text

categorizer component :

```
epochs = 2
with nlp.select_pipes(enable="textcat_multilabel"):
optimizer = nlp.resume_training()
for i in range(epochs):
random.shuffle(train_examples)
for example in train_examples:
nlp.update([example], sgd=optimizer)
```

Finally, we'll test how the text classifier component works for two example sentences :

```
doc2 = nlp("This is the best food I ever ate")
doc2.cats
{'POS': 0.9553419947624207, 'NEG': 0.061326123774051666}
```

```
4  doc3 = nlp("This food is so bad")
5  doc3.cats
6  {'POS': 0.21204468607902527, 'NEG': 0.8010350465774536}
```

Both the NEG and POS labels appear in the prediction result because we used the multilabel classifier. The results look good. The first sentence outputs a very high positive probability, and the second sentence is predicted as negative with a high probability.